

CAPGEMINI CODING SOLUTIONS MASTERBOOK

**185 Curated Past-Year Problems — Full Index + 48 Fully Worked Solutions
(Java)**

Companion to your Capgemini Exceller 2027 Prep Guide

Total problems in reference bank	185
High-priority (P1) problems	120
Good-to-have (P2) problems	65
Fully solved with Java code in this book	48 (spanning all 13 topics)
Topics covered	Arrays, Hashing, Strings, Linked List, Stack & Queues, Binary Search, Sorting, Binary Search Trees, Greedy, Dynamic Programming, Tries, Mathematics

How to Use This Book

This book has two parts. **Part 1** is the complete master index of all 185 problems from your reference sheet, grouped by topic, so you can see the full scope and look up any problem's difficulty/priority at a glance. **Part 2** gives fully worked solutions — problem statement, step-by-step approach, clean Java code, and time/space complexity — for 48 hand-picked problems covering every single topic, chosen to teach you the core pattern behind each category so you can solve the remaining unsolved problems in the index using the same techniques.

Priority legend: P1 = higher priority / more frequently reported — solve these first. P2 = good-to-have, solve if time permits.

Language note: Solutions are in Java since C/C++/Java are the commonly allowed languages in Capgemini's coding round (Python is often disallowed) — the logic translates directly to C++ if that's your preferred language.

How to practice each solved problem: Read the problem → try to solve it yourself for 10–15 minutes first → only then read the approach → then compare your code against the given solution → note any gap in your logic.

Part 1 — Complete Master Index (All 185 Problems)

Arrays — 34 problems in reference bank

#	Pri.	Diff.	Title	Status
1	P1	Medium	Majority Element – I	✓ Solved in Part 2
4	P1	Medium	Kadane's Algorithm (Maximum Subarray Sum)	✓ Solved in Part 2
10	P1	Medium	Next Permutation	✓ Solved in Part 2
13	P1	Medium	Maximum Product Subarray	—
23	P1	Hard	Merge Two Sorted Arrays Without Extra Space	—
30	P1	Medium	Count Subarrays with Given Sum	—
34	P1	Medium	Asteroid Collision	—
36	P1	Medium	Finding the Equilibrium Index in an Array	—
38	P1	Medium	Single Number – II	—
45	P1	Medium	Maximum Points You Can Obtain from Cards	—
55	P1	Hard	Celebrity Problem	—
58	P1	Hard	Build Heap from a Given Array	—
60	P1	Medium	3 Sum	✓ Solved in Part 2
62	P1	Hard	4 Sum	—
63	P1	Medium	Longest Subarray with Sum K	—
66	P1	Medium	Max Consecutive Ones – III	—
73	P1	Medium	3Sum Smaller	—
79	P1	Medium	Rearrange Array Elements by Sign	✓ Solved in Part 2
80	P2	Medium	Intersection of Two Sorted Arrays	—
81	P2	Medium	Move Zeros to End	—
82	P2	Medium	Remove Duplicates from Sorted Array	—
83	P2	Medium	Union of Two Sorted Arrays	—
91	P1	Hard	Candy Crush	—
110	P1	Medium	Dot Product of Two Sparse Vectors	—
115	P2	Medium	Reverse an Array – 2 (Reverse in Groups)	—
119	P2	Medium	Furthest Empty Plot from Buildings	—
120	P1	Medium	Faulty Sensor	—
125	P1	Hard	Flatten 2D Vector	—
126	P2	Medium	Intersection of Three Sorted Arrays	—
169	P1	Hard	Best Time to Buy and Sell Stock III	—
170	P1	Medium	Non-overlapping Intervals	—
175	P1	Medium	Find the Elements Removed from Nested Arrays ($A \supset B \supset C$)	—
176	P1	Medium	Longest Consecutive Path in a Grid	—
179	P1	Medium	Spiral Matrix Traversal	✓ Solved in Part 2

Hashing — 31 problems in reference bank

#	Pri.	Diff.	Title	Status
2	P1	Medium	Two Sum	✓ Solved in Part 2
11	P1	Medium	Longest Consecutive Sequence in an Array	✓ Solved in Part 2
46	P1	Medium	Binary Subarrays with Sum	—
67	P1	Medium	Majority Element – II	✓ Solved in Part 2
74	P1	Medium	Alt and Tab Simulation	—
78	P2	Medium	Set Difference of Two Arrays	—
90	P2	Easy	Calculate Frequency of Characters in a String	—
95	P2	Medium	Check if an Array Is a Subset of Another	—
105	P2	Medium	Count Prefix Appearances	—
106	P1	Hard	Count the Number of Good Subsequences	—
107	P1	Medium	Counting Elements	—
108	P2	Medium	Counting Frequencies of Array Elements	—
109	P1	Hard	Divide Array Into Increasing Sequences	—
111	P1	Medium	MEXX (Minimum Excluded Value)	—
112	P2	Easy	Highest Occurring Element in an Array	—
113	P2	Easy	Second Highest Occurring Element	—
114	P2	Medium	Sum of Highest and Lowest Frequency	—
116	P1	Medium	Isomorphic Strings	✓ Solved in Part 2
117	P1	Medium	Sort Characters by Frequency	—
118	P1	Medium	Valid Anagram	✓ Solved in Part 2
121	P1	Medium	Find All Non-Repeating Elements in an Array	—
122	P1	Medium	Find All Repeating Elements in an Array	—
123	P2	Medium	Find All Symmetric Pairs	—
124	P2	Medium	Find Distinct Expressions	—
127	P1	Medium	Line Reflection	—
128	P1	Hard	Match Alphanumerical Pattern in Matrix – I	—
129	P2	Medium	Minimum Unlocked Indices to Sort Nums	—
130	P1	Medium	Remove Duplicates from an Unsorted Linked List	—
131	P1	Medium	Remove Duplicates from an Unsorted Array	—
173	P2	Medium	Count Even Odd	—
174	P1	Medium	Group Anagrams	✓ Solved in Part 2

Strings — 26 problems in reference bank

#	Pri.	Diff.	Title	Status
3	P1	Medium	Longest Substring Without Repeating Characters	✓ Solved in Part 2
7	P1	Hard	Minimum Window Substring	✓ Solved in Part 2

#	Pri.	Diff.	Title	Status
40	P1	Medium	Number of Substrings Containing All Three Characters	—
54	P1	Hard	KMP Algorithm / LPS Array	—
56	P2	Hard	Rabin–Karp Algorithm	—
57	P2	Hard	Z Function	—
61	P1	Hard	Longest Repeating Character Replacement	✓ Solved in Part 2
64	P1	Medium	Longest Substring with At Most K Distinct Characters	—
72	P1	Hard	Shortest Palindrome	—
139	P2	Easy	Capitalize First and Last Character of Each Word	—
140	P2	Easy	Change Case of Each Character in a String	—
141	P2	Medium	Change Every Letter with the Next Lexicographic	—
142	P1	Medium	Check if String Is Decomposable Into Value-Equal Substrings	—
143	P2	Easy	Concatenate One String to Another	—
144	P2	Medium	Count Substrings with Only One Distinct Letter	—
145	P2	Easy	Count the Number of Words in a Given String	—
146	P2	Medium	Count Vowels, Consonants, and Spaces in a String	—
148	P2	Medium	Find Pattern in Infinite Stream II	—
149	P2	Easy	Find the ASCII Value of a Character	—
150	P2	Medium	Find the Largest Word in a String	—
151	P2	Medium	Flip Game	—
152	P2	Medium	Hexspeak	—
172	P1	Medium	One Away	—
177	P2	Easy	Move All '#' to the Front of a String	✓ Solved in Part 2
178	P2	Easy	Run-Length Encoding of a String	✓ Solved in Part 2
183	P2	Easy	Reverse a String	✓ Solved in Part 2

Linked List — 7 problems in reference bank

#	Pri.	Diff.	Title	Status
5	P1	Medium	Merge Two Sorted Lists	✓ Solved in Part 2
6	P1	Medium	Remove N-th Node from the End of a Linked List	✓ Solved in Part 2
17	P1	Medium	Clone a Linked List with Random and Next Pointers	—
18	P1	Medium	Add Two Numbers in Linked List	✓ Solved in Part 2
31	P1	Medium	Find the Intersection Point of a Y-Shaped Linked List	—
59	P1	Medium	Sort a Linked List of 0's, 1's and 2's	—
155	P1	Medium	Rotate a Linked List	✓ Solved in Part 2

Linked List (Design) — 2 problems in reference bank

#	Pri.	Diff.	Title	Status
8	P1	Medium	LRU Cache	✓ Solved in Part 2

#	Pri.	Diff.	Title	Status
51	P1	Hard	LFU Cache	—

Stack & Queues — 11 problems in reference bank

#	Pri.	Diff.	Title	Status
9	P1	Medium	Trapping Rain Water	✓ Solved in Part 2
19	P1	Hard	Largest Rectangle in a Histogram	—
20	P1	Hard	Sliding Window Maximum	✓ Solved in Part 2
22	P1	Medium	Implement Min Stack	✓ Solved in Part 2
32	P1	Medium	Next Greater Element	✓ Solved in Part 2
41	P1	Medium	Remove K Digits	—
42	P1	Medium	Sum of Subarray Minimums	—
65	P1	Medium	Next Greater Element – II	—
160	P1	Medium	Stock Span Problem	—
161	P1	Hard	Sum of Subarray Ranges	—
162	P1	Hard	Maximum Rectangles	—

Binary Search — 15 problems in reference bank

#	Pri.	Diff.	Title	Status
12	P1	Medium	Longest Increasing Subsequence	—
24	P1	Medium	Search in a 2D Matrix – II	✓ Solved in Part 2
25	P1	Medium	Find Peak Element	✓ Solved in Part 2
26	P1	Hard	Median of Two Sorted Arrays	✓ Solved in Part 2
35	P1	Medium	Single Element in a Sorted Array	—
44	P1	Medium	Find Out How Many Times the Array Is Rotated	—
47	P1	Hard	Matrix Median	—
68	P1	Hard	Kth Element of Two Sorted Arrays	—
70	P1	Hard	Find Peak Element – II	—
76	P2	Medium	Array Upper Bound	—
85	P1	Medium	Find Row with Maximum 1's	—
86	P1	Medium	Search in a 2D Matrix	—
87	P2	Medium	Lower Bound	—
88	P2	Medium	Search X in a Sorted Array	✓ Solved in Part 2
89	P2	Medium	Upper Bound	—

Sorting — 7 problems in reference bank

#	Pri.	Diff.	Title	Status
14	P1	Medium	Sort an Array of 0's, 1's and 2's	✓ Solved in Part 2
48	P1	Medium	Find the Repeating and Missing Number	✓ Solved in Part 2

#	Pri.	Diff.	Title	Status
132	P1	Medium	Analyze User Website Visit Pattern	—
133	P1	Medium	Before and After Puzzle	—
134	P1	Medium	Best Meeting Point	—
136	P2	Medium	Better Compression of String	—
138	P1	Medium	Campus Bikes	—

Binary Search Trees — 13 problems in reference bank

#	Pri.	Diff.	Title	Status
15	P1	Medium	Check if a Binary Tree is a BST	✓ Solved in Part 2
27	P1	Medium	Kth Smallest and Kth Largest Element in a BST	✓ Solved in Part 2
37	P1	Medium	Lowest Common Ancestor in a BST	✓ Solved in Part 2
43	P1	Medium	Two Sum in a BST	—
49	P1	Medium	Correct a BST with Two Nodes Swapped	—
53	P1	Hard	Largest BST in a Binary Tree	—
154	P2	Medium	Range Sum in BST	—
156	P1	Medium	Delete a Node in BST	—
157	P1	Medium	Inorder Successor and Predecessor in BST	—
158	P2	Medium	Floor and Ceil in a BST	—
164	P2	Medium	Search in BST	—
165	P2	Medium	Insert a Given Node in BST	—
166	P1	Hard	Construct a BST from a Preorder Traversal	—

Greedy Algorithms — 6 problems in reference bank

#	Pri.	Diff.	Title	Status
16	P1	Medium	Jump Game – I	✓ Solved in Part 2
28	P1	Hard	Candy	—
29	P1	Medium	Valid Parenthesis Checker (with Wildcard)	✓ Solved in Part 2
33	P1	Medium	Meeting Rooms	✓ Solved in Part 2
50	P1	Hard	Job Sequencing Problem	—
147	P2	Medium	Count Pairs of Equal Substrings with Minimum Difference	—

Dynamic Programming — 5 problems in reference bank

#	Pri.	Diff.	Title	Status
39	P1	Medium	Best Time to Buy and Sell Stock – IV	—
69	P1	Medium	Best Time to Buy and Sell Stock with Transaction Fee	—
135	P1	Medium	Best Time to Buy and Sell Stock with Cooldown	—
167	P1	Medium	Best Time to Buy and Sell Stock	✓ Solved in Part 2
168	P1	Medium	Best Time to Buy and Sell Stock II	✓ Solved in Part 2

Tries — 6 problems in reference bank

#	Pri.	Diff.	Title	Status
52	P1	Hard	Trie Implementation and Advanced Operations	✓ Solved in Part 2
71	P1	Hard	Longest Word with All Prefixes	—
137	P1	Medium	Bold Words in String	—
153	P1	Hard	Number of Distinct Substrings in a String	—
163	P1	Hard	Distinct Substrings	—
171	P1	Medium	Longest Common Prefix	✓ Solved in Part 2

Mathematics — 21 problems in reference bank

#	Pri.	Diff.	Title	Status
75	P2	Easy	Armstrong Number	✓ Solved in Part 2
77	P2	Easy	Number of Jumps	—
84	P2	Easy	Automorphic Number	—
92	P2	Easy	Check if a Number is a Harshad (Niven) Number	—
93	P2	Easy	Check if a Number is a Strong Number	—
94	P2	Easy	Check if a Number is an Abundant Number	—
96	P2	Easy	Check Whether a Floating-Point Number Is Even or Odd	—
97	P2	Easy	Check Whether a Given Number Is Even or Odd	✓ Solved in Part 2
98	P2	Easy	Check Whether a Number Is Positive or Negative	—
99	P2	Easy	Confusing Number	—
100	P2	Easy	Convert a Number to Hexadecimal	—
101	P2	Easy	Convert Decimal to Binary Number	—
102	P2	Easy	Convert Decimal to Octal Number	—
103	P1	Easy	Convert Digits / Numbers to Words	—
104	P2	Hard	Convex Polygon	—
159	P1	Medium	Count Number of Nice Subarrays	—
180	P2	Medium	Evaluate the Polynomial $a^3 + a^2b + 2a^2b + 2ab^2 + ab^2 + b^3$	—
181	P2	Easy	Dealership Tyres	✓ Solved in Part 2
182	P2	Easy	Print Palindrome Numbers in a Range	—
184	P2	Easy	Fibonacci Series	✓ Solved in Part 2
185	P2	Easy	Swap Two Numbers Without a Temporary Variable	✓ Solved in Part 2

Hashing (Trees) — 1 problems in reference bank

#	Pri.	Diff.	Title	Status
21	P1	Medium	Construct a Binary Tree from Preorder and Inorder Traversals	—

Part 2 — Fully Worked Solutions (48 Problems)

Every topic from the index above is represented here with at least 2–6 fully solved problems, chosen to cover the most important pattern in that category. Work through them in order within each topic — later problems often reuse the pattern from earlier ones.

ARRAYS — 6 problems in reference bank

#1. Majority Element – I

#1 P1 Medium Arrays

Problem: Given an array of n integers, find the majority element: the value that appears strictly more than $n/2$ times. You may assume that the array always contains such an element. Your solution should ideally run in linear time and constant extra space, for example using the Boyer–Moore voting technique, rather than sorting the array or storing counts in a hash map. Print the majority element. This problem checks whether you can spot that a true majority element survives repeated cancellation against different elements, which makes a single pass with one candidate and one counter sufficient. Read the input from standard input and print only the required value.

Approach

- This is the Boyer–Moore Voting Algorithm.
- Keep a 'candidate' and a 'count'. Walk through the array once.
- If count is 0, pick the current element as the new candidate.
- If the current element equals the candidate, increment count; otherwise decrement it.
- Because the majority element occurs more than $n/2$ times, it always survives this cancellation process.

Solution (Java)

```
public int majorityElement(int[] nums) {
    int count = 0, candidate = 0;
    for (int num : nums) {
        if (count == 0) candidate = num;
        count += (num == candidate) ? 1 : -1;
    }
    return candidate;
}
```

Time: $O(n)$ Space: $O(1)$

#4. Kadane's Algorithm (Maximum Subarray Sum)

#4 P1 Medium Arrays

Problem: Given an integer array that may contain negative numbers, find the contiguous non-empty subarray that has the largest possible sum and print that sum. A brute force method that checks all subarrays needs cubic or quadratic time, but Kadane's algorithm solves the task in one pass by keeping the best sum of a subarray ending at the current position, which is either the current element alone or the current element added to the previous best. Remember that the chosen subarray must contain at least one element, so when every value is negative the answer is the largest single element, not zero.

Approach

- Kadane's Algorithm: track the best sum of a subarray that ENDS at the current index.
- At each step, either extend the previous subarray (add current element) or start fresh from the current element — whichever is larger.
- Keep a running 'best overall' answer as you go.

Solution (Java)

```
public int maxSubArray(int[] nums) {
    int maxEndingHere = nums[0], maxSoFar = nums[0];
    for (int i = 1; i < nums.length; i++) {
        maxEndingHere = Math.max(nums[i], maxEndingHere + nums[i]);
        maxSoFar = Math.max(maxSoFar, maxEndingHere);
    }
    return maxSoFar;
}
```

Time: $O(n)$ Space: $O(1)$

#10. Next Permutation

#10

P1

Medium

Arrays

Problem: A permutation of an array is one of its possible orderings. The next permutation is the next lexicographically greater ordering of the same numbers. If the given arrangement is already the largest possible, wrap around and return the smallest arrangement, which is the array sorted in ascending order. Rearrange the elements to obtain the next permutation and print the result. The array may contain duplicates. An efficient algorithm finds the rightmost position whose value is smaller than its successor, swaps it with the smallest larger value to its right, and then reverses the suffix after that position. It works in linear time and constant space.

Approach

- Scan from the right to find the first index i where $\text{nums}[i] < \text{nums}[i+1]$ (the 'pivot').
- If no such i exists, the array is in descending order — reverse the whole array (wrap-around case).
- Otherwise, scan from the right again to find the first index $j > i$ where $\text{nums}[j] > \text{nums}[i]$, and swap $\text{nums}[i]$ and $\text{nums}[j]$.
- Finally, reverse the suffix starting right after index i to get the smallest possible ordering for that tail.

Solution (Java)

```
public void nextPermutation(int[] nums) {
    int n = nums.length, i = n - 2;
    while (i >= 0 && nums[i] >= nums[i + 1]) i--;
    if (i >= 0) {
        int j = n - 1;
        while (nums[j] <= nums[i]) j--;
        int t = nums[i]; nums[i] = nums[j]; nums[j] = t;
    }
    // reverse suffix from i+1 to end
    int l = i + 1, r = n - 1;
    while (l < r) { int t = nums[l]; nums[l] = nums[r]; nums[r] = t; l++; r--; }
}
```

Time: $O(n)$ **Space:** $O(1)$

#60. 3 Sum

#60

P1

Medium

Arrays

Problem: Given an integer array, find all unique triplets of values (a, b, c) such that $a + b + c = 0$. The three numbers must come from three different positions of the array, but the answer must not contain the same triplet of values twice, even if it can be formed from different positions. Print each triplet in non-decreasing order on its own line, and sort the triplets lexicographically. If no triplet exists, print []. The standard approach sorts the array, fixes the first element, and uses two pointers on the rest of the array while skipping duplicate values, giving quadratic time and no extra hash sets.

Approach

- Sort the array first.
- Fix one element with index i , then use two pointers (lo, hi) on the remaining subarray to find pairs that sum to $-\text{nums}[i]$.
- Skip duplicate values for i, lo , and hi to avoid duplicate triplets in the answer.

Solution (Java)

```

public List<List<Integer>> threeSum(int[] nums) {
    Arrays.sort(nums);
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < nums.length - 2; i++) {
        if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicate i
        int lo = i + 1, hi = nums.length - 1;
        while (lo < hi) {
            int sum = nums[i] + nums[lo] + nums[hi];
            if (sum == 0) {
                res.add(Arrays.asList(nums[i], nums[lo], nums[hi]));
                while (lo < hi && nums[lo] == nums[lo + 1]) lo++;
                while (lo < hi && nums[hi] == nums[hi - 1]) hi--;
                lo++; hi--;
            } else if (sum < 0) lo++;
            else hi--;
        }
    }
    return res;
}

```

Time: $O(n^2)$ Space: $O(1)$ extra (excluding output)

#79. Rearrange Array Elements by Sign

#79 P1 Medium Arrays

Problem: You are given an array of even length containing an equal number of positive and negative integers, and no zeros. Rearrange the elements so that positive and negative numbers alternate, starting with a positive number. The relative order of the positive numbers among themselves must stay the same as in the original array, and the same is required for the negative numbers. Print the rearranged array. A straightforward method uses two separate lists, and a more space efficient variant fills a result array using two write positions, one at even indices for positive values and one at odd indices for negative values, in a single pass over the input.

Approach

- The array has an equal number of positives and negatives; result must alternate starting with a positive.
- Create a result array. Use one pointer for even (positive) slots and one for odd (negative) slots.
- Walk through the original array once: place positives at the next even slot, negatives at the next odd slot.

Solution (Java)

```

public int[] rearrangeArray(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    int pos = 0, neg = 1;
    for (int num : nums) {
        if (num > 0) { res[pos] = num; pos += 2; }
        else { res[neg] = num; neg += 2; }
    }
    return res;
}

```

Time: $O(n)$ Space: $O(n)$ for the output array

#179. Spiral Matrix Traversal

#179 P1 Medium Arrays

Problem: You are given a matrix with m rows and n columns. Print all of its elements in spiral order, starting from the top-left cell, moving right along the first row, then down the last column, then left along the last row, then up the first column, and continue spiralling inwards until every element has been printed exactly once. The matrix does not have to be square, so single rows, single columns and rectangles must be handled correctly. Maintain four boundaries: top, bottom, left and right. After traversing a side, shrink the corresponding boundary. Before traversing the bottom row and the left column, check that the boundaries have not crossed, otherwise elements would be printed twice.

Approach

- Maintain four boundaries: top, bottom, left, right.
- Traverse right along the top row, down the right column, left along the bottom row, up the left column — then shrink each boundary inward.
- Stop when $top > bottom$ or $left > right$.

Solution (Java)

```
public List<Integer> spiralOrder(int[][] m) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.add(m[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) res.add(m[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.add(m[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.add(m[i][left]);
            left++;
        }
    }
    return res;
}
```

Time: $O(m \cdot n)$ Space: $O(1)$ extra (excluding output)

HASHING — 6 problems in reference bank

#2. Two Sum

#2	P1	Medium	Hashing
----	----	--------	---------

Problem: You are given an array of integers and a target value. Find two different positions i and j such that the elements stored at those positions add up to the target, and print the two indices in increasing order using zero-based indexing. It is guaranteed that exactly one valid pair exists, and you may not use the same element twice. A brute force check of every pair takes quadratic time, so aim for a single pass that stores each value seen so far in a hash map together with its index, and looks up the needed complement before inserting the current element.

Approach

- Use a HashMap that stores value $\rightarrow$ index as you scan left to right.
- For each number, first check if $(\text{target} - \text{number})$ already exists in the map — if so, you found your pair.
- Otherwise insert the current number and its index, then continue.

Solution (Java)

```
public int[] twoSum(int[] nums, int target) {
    Map<Integer, Integer> seen = new HashMap<>();
    for (int i = 0; i < nums.length; i++) {
        int need = target - nums[i];
        if (seen.containsKey(need)) return new int[]{seen.get(need), i};
        seen.put(nums[i], i);
    }
    return new int[]{-1, -1}; // no valid pair (per constraints, shouldn't happen)
}
```

Time: $O(n)$ **Space:** $O(n)$

#11. Longest Consecutive Sequence in an Array

#11	P1	Medium	Hashing
-----	----	--------	---------

Problem: Given an unsorted array of integers, find the length of the longest sequence of consecutive values that can be formed using elements of the array. The elements do not need to be adjacent in the array, and their order in the input does not matter, so for instance 4, 2, 3, 1 forms the sequence 1, 2, 3, 4. Duplicates are counted only once. Sorting would take $O(n \log n)$ time. To achieve $O(n)$, place all numbers in a hash set and start counting only from those numbers whose predecessor is not present. Print the length of the longest run.

Approach

- Put every number into a HashSet for $O(1)$ lookups.
- For each number, only start counting a streak if $(\text{number} - 1)$ is NOT in the set — this guarantees you only start from the beginning of a run, keeping total work linear.
- From each valid start, keep checking $\text{number}+1$, $\text{number}+2$, ... to measure the streak length.

Solution (Java)

```
public int longestConsecutive(int[] nums) {
    Set<Integer> set = new HashSet<>();
    for (int n : nums) set.add(n);
    int best = 0;
    for (int n : set) {
        if (!set.contains(n - 1)) { // start of a streak
            int len = 1, cur = n;
            while (set.contains(cur + 1)) { cur++; len++; }
            best = Math.max(best, len);
        }
    }
    return best;
}
```

Time: O(n) Space: O(n)

#118. Valid Anagram

#118 P1 Medium Hashing

Problem: Given two strings *s* and *t*, determine whether *t* is an anagram of *s*. Two strings are anagrams when they contain exactly the same characters with exactly the same frequencies, possibly in a different order. The comparison is case sensitive, and strings of different lengths cannot be anagrams. Sorting both strings and comparing them works in $O(n \log n)$ time. A faster approach uses a frequency array of 26 integers for lowercase letters or a hash map for general characters: increase the counters while scanning *s*, decrease them while scanning *t*, and verify that all counters return to zero. Print true if *t* is an anagram of *s* and false otherwise.

Approach

- Two strings are anagrams only if they have the same length and the same character frequencies.
- Use a fixed-size count array (26 for lowercase letters): increment for each character of *s*, decrement for each character of *t*.
- If every count returns to zero, they are anagrams.

Solution (Java)

```
public boolean isAnagram(String s, String t) {
    if (s.length() != t.length()) return false;
    int[] cnt = new int[26];
    for (int i = 0; i < s.length(); i++) {
        cnt[s.charAt(i) - 'a']++;
        cnt[t.charAt(i) - 'a']--;
    }
    for (int c : cnt) if (c != 0) return false;
    return true;
}
```

Time: O(n) Space: O(1) (fixed 26-size array)

#174. Group Anagrams

#174 P1 Medium Hashing

Problem: Given an array of strings, group the anagrams together. Two strings are anagrams if one can be formed by rearranging the letters of the other, using every letter exactly once. For example, eat, tea and ate are anagrams of each other. Print each group on its own line, with the words of the group separated by spaces in their original order. The groups must be printed in the order in which their first word appears in the input. Compute a key for every word by sorting its letters, or by counting letter frequencies, store the words in a hash map that maps each key to a list, and finally print the lists. The time complexity is $O(n \cdot k \log k)$ for *n* words of length *k*.

Approach

- Anagrams share the same sorted form (e.g. 'eat' and 'tea' both sort to 'aet').
- Use a HashMap where the key is the sorted version of each word and the value is the list of original words sharing that key.

Solution (Java)

```
public List<List<String>> groupAnagrams(String[] strs) {
    Map<String, List<String>> map = new HashMap<>();
    for (String s : strs) {
        char[] c = s.toCharArray();
        Arrays.sort(c);
        String key = new String(c);
        map.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
    }
    return new ArrayList<>(map.values());
}
```

Time: $O(n \cdot k \log k)$ (*n* words, *k* = max word length) Space: $O(n \cdot k)$

#67. Majority Element – II

#67

P1

Medium

Hashing

Problem: Given an integer array of size n , find all elements that appear strictly more than $n/3$ times, where the division is an integer division rounded down. There can be at most two such elements. Print them in increasing order on one line, or print [] if none exist. A hash map of counts solves the task in linear time and linear space, but the extended Boyer–Moore voting algorithm reduces the space to $O(1)$: keep two candidates with two counters, decrease both counters when a third distinct value appears, and finally verify the two candidates by counting their actual occurrences in a second pass through the array.

Approach

- Extended Boyer–Moore Voting: since more than $n/3$ means at most 2 such elements can exist, track TWO candidates and two counters.
- First pass: update candidate1/candidate2 and their counts using the same cancellation logic as the single-candidate version.
- Second pass: verify both candidates actually occur more than $n/3$ times (the first pass alone can give false positives).

Solution (Java)

```
public List<Integer> majorityElement(int[] nums) {
    int cand1 = 0, cand2 = 1, cnt1 = 0, cnt2 = 0;
    for (int n : nums) {
        if (n == cand1) cnt1++;
        else if (n == cand2) cnt2++;
        else if (cnt1 == 0) { cand1 = n; cnt1 = 1; }
        else if (cnt2 == 0) { cand2 = n; cnt2 = 1; }
        else { cnt1--; cnt2--; }
    }
    cnt1 = 0; cnt2 = 0;
    for (int n : nums) { if (n == cand1) cnt1++; else if (n == cand2) cnt2++; }
    List<Integer> res = new ArrayList<>();
    if (cnt1 > nums.length / 3) res.add(cand1);
    if (cnt2 > nums.length / 3) res.add(cand2);
    return res;
}
```

Time: $O(n)$ **Space:** $O(1)$

#116. Isomorphic Strings

#116

P1

Medium

Hashing

Problem: Two strings s and t are isomorphic if the characters of s can be replaced by characters of t to obtain t . All occurrences of a character must be replaced with the same character, and the mapping must be one-to-one, so two different characters of s cannot map to the same character of t . A character is allowed to map to itself. Given the strings, print true if they are isomorphic and false otherwise. Strings of different lengths can never be isomorphic. Maintain two hash maps, one from s to t and one from t to s , and check both directions for consistency at every position, or compare the index of the first occurrence of each character.

Approach

- Two strings are isomorphic if there's a consistent one-to-one character mapping both ways ($s \rightarrow t$ and $t \rightarrow s$).
- Use two hashmaps: one mapping each character of s to its paired character in t , and one the reverse.
- At every position, if a mapping already exists it must match the current pair; if not, create both mappings.

Solution (Java)

```
public boolean isIsomorphic(String s, String t) {
    if (s.length() != t.length()) return false;
    Map<Character, Character> mapST = new HashMap<>();
    Map<Character, Character> mapTS = new HashMap<>();
    for (int i = 0; i < s.length(); i++) {
        char a = s.charAt(i), b = t.charAt(i);
        if (mapST.containsKey(a) && mapST.get(a) != b) return false;
        if (mapTS.containsKey(b) && mapTS.get(b) != a) return false;
        mapST.put(a, b);
        mapTS.put(b, a);
    }
    return true;
}
```

Time: $O(n)$ Space: $O(1)$ (at most 256 characters)

STRINGS — 6 problems in reference bank

#3. Longest Substring Without Repeating Characters

#3

P1

Medium

Strings

Problem: Given a string *s*, determine the length of the longest contiguous substring that contains no repeated character. A substring must be made of consecutive characters of the original string, so a subsequence such as removing letters from the middle is not allowed. The classic approach uses a sliding window with two pointers and a map from character to its most recent index, so that whenever a duplicate enters the window, the left boundary jumps just past the previous occurrence. This gives a linear time solution. Print a single integer, the maximum window length that was ever seen while scanning the whole string.

Approach

- Sliding window with a HashMap storing the last seen index of each character.
- Expand the window with the right pointer; if the current character was seen before AND its last index is inside the current window, jump the left pointer to just past that occurrence.
- Track the maximum window size seen at every step.

Solution (Java)

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> lastIdx = new HashMap<>();
    int left = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (lastIdx.containsKey(c) && lastIdx.get(c) >= left) {
            left = lastIdx.get(c) + 1;
        }
        lastIdx.put(c, right);
        best = Math.max(best, right - left + 1);
    }
    return best;
}
```

Time: $O(n)$ **Space:** $O(\min(n, \text{charset size}))$

#7. Minimum Window Substring

#7

P1

Hard

Strings

Problem: Given two strings *s* and *t*, find the smallest substring of *s* that contains every character of *t*, including duplicates. If *t* contains two letters 'a', the window must also contain at least two letters 'a'. If several windows share the minimum length, print the one that starts earliest. If no such window exists, print -1. The standard method is a sliding window: expand the right end until all required characters are covered, then shrink the left end as far as possible while keeping the window valid, recording the best window seen. Frequency counts for the characters of *t* make each step constant time.

Approach

- Classic 'expand-then-shrink' sliding window.
- Build a frequency map of characters needed from *t*. Expand the right pointer until the window contains every needed character (track how many distinct required characters are currently satisfied).
- Once valid, shrink from the left as far as possible while staying valid, recording the smallest valid window seen.
- Continue expanding right and repeating until the end of *s*.

Solution (Java)

```

public String minWindow(String s, String t) {
    if (s.isEmpty() || t.isEmpty()) return "";
    Map<Character, Integer> need = new HashMap<>();
    for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);
    Map<Character, Integer> window = new HashMap<>();
    int required = need.size(), formed = 0;
    int left = 0, bestLen = Integer.MAX_VALUE, bestStart = 0;
    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        window.merge(c, 1, Integer::sum);
        if (need.containsKey(c) && window.get(c).intValue() == need.get(c).intValue()) formed++;
        while (formed == required) {
            if (right - left + 1 < bestLen) { bestLen = right - left + 1; bestStart = left; }
            char lc = s.charAt(left);
            window.put(lc, window.get(lc) - 1);
            if (need.containsKey(lc) && window.get(lc) < need.get(lc)) formed--;
            left++;
        }
    }
    return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestStart, bestStart + bestLen);
}

```

Time: $O(|s| + |t|)$ **Space:** $O(|s| + |t|)$

#61. Longest Repeating Character Replacement

#61	P1	Hard	Strings
-----	----	------	---------

Problem: You are given a string of uppercase English letters and an integer k . In one operation you may choose any character of the string and change it to any other uppercase letter, and you may perform at most k such operations. Find the length of the longest substring that can be made to contain a single repeated letter after the operations. Use a sliding window: the window is valid while its length minus the count of its most frequent letter does not exceed k , meaning the other letters can be replaced. Expand the right end and move the left end when the window becomes invalid. Print the maximum valid window length.

Approach

- Sliding window that tracks the count of the most frequent character currently inside the window.
- A window of length L is 'fixable' with at most k replacements if $(L - \text{maxFreqInWindow}) \leq k$.
- Expand right always; if the window becomes invalid, shrink left by one (the window size never needs to shrink below its historical best, so this simple rule still finds the correct maximum length).

Solution (Java)

```

public int characterReplacement(String s, int k) {
    int[] cnt = new int[26];
    int left = 0, maxFreq = 0, best = 0;
    for (int right = 0; right < s.length(); right++) {
        maxFreq = Math.max(maxFreq, ++cnt[s.charAt(right) - 'A']);
        if (right - left + 1 - maxFreq > k) {
            cnt[s.charAt(left) - 'A']--;
            left++;
        }
        best = Math.max(best, right - left + 1);
    }
    return best;
}

```

Time: $O(n)$ **Space:** $O(1)$ (fixed 26-size array)

#178. Run-Length Encoding of a String

#178	P2	Easy	Strings
------	----	------	---------

Problem: In the online test the student is given a string in which several characters are repeated consecutively. The task is to reduce the size of the string using a simple mathematical logic: every maximal group of the same character is

replaced by the character followed by the number of times it occurs in the group. For example, aabbbbbeeeffggg is encoded as a2b4e4f2g3. A character that occurs once is written with the count 1, so abc is encoded as a1b1c1. Only consecutive equal characters are grouped, therefore aabaa becomes a2b1a2. Scan the string once, keep the current character and its count, and append the pair to the answer whenever the character changes or the string ends.

Approach

- Single pass: keep a counter for the current run of identical consecutive characters.
- Whenever the next character differs, flush the current character and its count (skip printing the count if it's 1) and reset the counter.

Solution (Java)

```
public String compress(String s) {
    StringBuilder sb = new StringBuilder();
    int i = 0, n = s.length();
    while (i < n) {
        char c = s.charAt(i);
        int j = i;
        while (j < n && s.charAt(j) == c) j++;
        int count = j - i;
        sb.append(c);
        if (count > 1) sb.append(count);
        i = j;
    }
    return sb.toString();
}
```

Time: $O(n)$ Space: $O(n)$ for the output

#177. Move All '#' to the Front of a String

#177

P2

Easy

Strings

Problem: You are given a string that contains the character # in some positions together with other characters. Move all the # characters to the front of the string, so that they come first, while keeping the relative order of all the other characters exactly the same as in the input. The length of the string does not change. For example, the string Ab#c#d becomes ##Abcd. An efficient in-place style solution scans the string from right to left with two pointers: copy each non-# character towards the end of the array using a write pointer, then fill the remaining positions at the front with #. A simpler approach counts the number of # characters, builds a new string of hashes, and appends the other characters in order.

Approach

- Single pass, two buffers: count how many '#' characters appear, and separately build a string of only the non-# characters in their original order.
- The answer is simply that many '#' characters followed by the collected letters.

Solution (Java)

```
public String moveHashToFront(String s) {
    StringBuilder letters = new StringBuilder();
    int hashCount = 0;
    for (char c : s.toCharArray()) {
        if (c == '#') hashCount++;
        else letters.append(c);
    }
    StringBuilder res = new StringBuilder();
    for (int i = 0; i < hashCount; i++) res.append('#');
    res.append(letters);
    return res.toString();
}
```

Time: $O(n)$ Space: $O(n)$

#183. Reverse a String

Problem: Given a string, print the string in reverse order, so that the last character becomes the first, the second last becomes the second, and so on. For example, the string Capgemini becomes inimegpaC. The reversal must not change the characters themselves; only their positions change, and the case of every letter is kept. In an interview the built-in reverse function is usually not allowed, so the expected method uses two pointers: place one pointer at the start and one at the end, swap the characters they point to, move the pointers towards each other, and stop when they meet in the middle. This runs in linear time and needs constant extra space if the string can be modified in place.

Approach

- Classic two-pointer in-place swap: one pointer at the start, one at the end.
- Swap the characters and move both pointers toward the centre until they meet.

Solution (Java)

```
public void reverseString(char[] s) {  
    int l = 0, r = s.length - 1;  
    while (l < r) {  
        char t = s[l]; s[l] = s[r]; s[r] = t;  
        l++; r--;  
    }  
}
```

Time: $O(n)$ Space: $O(1)$

LINKED LIST — 4 problems in reference bank

#5. Merge Two Sorted Lists

#5 P1 Medium Linked List

Problem: You are given the heads of two singly linked lists whose values are already sorted in non-decreasing order. Merge them into one sorted linked list by splicing together the existing nodes and print the values of the merged list from head to tail. Either list may be empty, and duplicate values are allowed. The efficient method walks along both lists with two pointers, always attaching the smaller front node to the result, and finally attaches whatever remains of the non-empty list. This takes time proportional to the total number of nodes. If the merged list is empty, print [].

Approach

- Use a dummy head node so you don't need special-case handling for the very first node.
- Walk both lists with two pointers; always attach the smaller front node to the result and advance that pointer.
- When one list runs out, attach whatever remains of the other list.

Solution (Java)

```
public ListNode mergeTwoLists(ListNode a, ListNode b) {
    ListNode dummy = new ListNode(-1);
    ListNode cur = dummy;
    while (a != null && b != null) {
        if (a.val <= b.val) { cur.next = a; a = a.next; }
        else { cur.next = b; b = b.next; }
        cur = cur.next;
    }
    cur.next = (a != null) ? a : b;
    return dummy.next;
}
```

Time: $O(n + m)$ Space: $O(1)$ extra (reuses existing nodes)

#6. Remove N-th Node from the End of a Linked List

#6 P1 Medium Linked List

Problem: Given the head of a singly linked list and an integer k, delete the k-th node counted from the end of the list, then print the remaining list from head to tail. The value k is always valid, meaning it does not exceed the length of the list. A neat one-pass solution uses two pointers: move the fast pointer k steps ahead, then advance both pointers together until the fast one reaches the last node, at which point the slow pointer sits just before the node that must be removed. A dummy head node makes the case of removing the first node easy. Print [] if the list becomes empty.

Approach

- Two-pointer trick with a dummy head (handles removing the very first node cleanly).
- Move a 'fast' pointer k+1 steps ahead of a 'slow' pointer.
- Then advance both together until fast reaches the end — slow now sits exactly before the node to delete.

Solution (Java)

```
public ListNode removeNthFromEnd(ListNode head, int k) {
    ListNode dummy = new ListNode(0);
    dummy.next = head;
    ListNode fast = dummy, slow = dummy;
    for (int i = 0; i < k + 1; i++) fast = fast.next;
    while (fast != null) { fast = fast.next; slow = slow.next; }
    slow.next = slow.next.next;
    return dummy.next;
}
```

Time: $O(n)$ Space: $O(1)$ (single pass)

#18. Add Two Numbers in Linked List

#18

P1

Medium

Linked List

Problem: Two non-negative integers are stored in two linked lists, where each node holds a single digit and the digits are stored in reverse order, so the head holds the ones digit. Add the two numbers and print the sum in the same representation, again with the least significant digit first. The lists may have different lengths, and a final carry can create one extra digit, such as 5 + 5 giving 0 followed by 1. Walk both lists together, keep a running carry, and append one digit for each step. Neither number has leading zeros except the number zero itself.

Approach

- Digits are stored in reverse order in each list, so add like elementary-school addition, from the least-significant digit.
- Walk both lists together, adding corresponding digits plus any carry, and create a new node for each resulting digit.
- Keep going while either list still has nodes, or a carry remains.

Solution (Java)

```
public ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(0);
    ListNode cur = dummy;
    int carry = 0;
    while (l1 != null || l2 != null || carry != 0) {
        int x = (l1 != null) ? l1.val : 0;
        int y = (l2 != null) ? l2.val : 0;
        int sum = x + y + carry;
        carry = sum / 10;
        cur.next = new ListNode(sum % 10);
        cur = cur.next;
        if (l1 != null) l1 = l1.next;
        if (l2 != null) l2 = l2.next;
    }
    return dummy.next;
}
```

Time: $O(\max(n, m))$ **Space:** $O(\max(n, m))$ for the result list

#155. Rotate a Linked List

#155

P1

Medium

Linked List

Problem: Given the head of a singly linked list and an integer k, rotate the list to the right by k places, so that the last k nodes move to the front while the relative order inside both parts stays the same. The value of k can be larger than the length of the list, in which case rotating by k is the same as rotating by k modulo the length. For the list 1 -> 2 -> 3 -> 4 -> 5 and k = 2, the answer is 4 -> 5 -> 1 -> 2 -> 3. Join the tail to the head to form a circle, walk length minus k steps to the new tail, and cut the circle after it.

Approach

- First find the list's length and connect the tail to the head to make it circular.
- Effective rotation is $k \% \text{length}$ (rotating by the full length changes nothing).
- The new tail is at position $(\text{length} - k \% \text{length} - 1)$ from the old head; break the circle right after it to get the new head.

Solution (Java)

```
public ListNode rotateRight(ListNode head, int k) {
    if (head == null || head.next == null || k == 0) return head;
    int len = 1;
    ListNode tail = head;
    while (tail.next != null) { tail = tail.next; len++; }
    tail.next = head;                // make circular
    int stepsToNewTail = len - (k % len);
    ListNode newTail = head;
    for (int i = 1; i < stepsToNewTail; i++) newTail = newTail.next;
    ListNode newHead = newTail.next;
    newTail.next = null;             // break the circle
    return newHead;
}
```

Time: $O(n)$ Space: $O(1)$

LINKED LIST (DESIGN) — 1 problems in reference bank

#8. LRU Cache

#8

P1

Medium

Linked List (Design)

Problem: Design a data structure that behaves as a Least Recently Used cache with a fixed capacity. It must support `put(key, value)`, which inserts or updates a pair, and `get(key)`, which returns the stored value or -1 when the key is absent. Both operations must run in average constant time, which is normally achieved with a hash map combined with a doubly linked list ordered by recency. Reading or writing a key makes it the most recently used one. When an insertion would exceed the capacity, evict the least recently used key. Process the given operations in order and print the result of every `get` operation on its own line.

Approach

- Needs $O(1)$ average `get/put` with recency ordering. Simplest correct approach: Java's `LinkedHashMap` with `accessOrder=true`, overriding `removeEldestEntry` to evict once capacity is exceeded.
- (In an interview, also be ready to explain the manual approach: a `HashMap` plus a hand-built doubly linked list, where `get()/put()` move the accessed node to the front and eviction removes the tail.)

Solution (Java)

```
class LRUCache extends LinkedHashMap<Integer, Integer> {
    private int capacity;
    public LRUCache(int capacity) {
        super(capacity, 0.75f, true);    // true = access order
        this.capacity = capacity;
    }
    public int get(int key) {
        return super.getOrDefault(key, -1);
    }
    public void put(int key, int value) {
        super.put(key, value);
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
        return size() > capacity;
    }
}
```

Time: $O(1)$ average for `get/put` Space: $O(\text{capacity})$

STACK & QUEUES — 4 problems in reference bank

#9. Trapping Rain Water

#9 P1 Medium Stack & Queues

Problem: You are given n non-negative integers that represent an elevation map in which every bar has width one. After rain, water gets trapped in the gaps between bars. Compute the total volume of water that can be stored. The water above a position is limited by the smaller of the tallest bar to its left and the tallest bar to its right, minus the height of the bar itself. You can compute this using prefix and suffix maxima, a monotonic stack, or a two pointer technique in constant extra space. Print a single integer, the total trapped water.

Approach

- Two-pointer approach from both ends. Track leftMax and rightMax seen so far.
- At each step, move the pointer on the side with the smaller max inward — water trapped at that position is (smaller side's max – current height), which is guaranteed correct because the other side's max is at least as large.

Solution (Java)

```
public int trap(int[] h) {
    int left = 0, right = h.length - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left < right) {
        if (h[left] <= h[right]) {
            leftMax = Math.max(leftMax, h[left]);
            water += leftMax - h[left];
            left++;
        } else {
            rightMax = Math.max(rightMax, h[right]);
            water += rightMax - h[right];
            right--;
        }
    }
    return water;
}
```

Time: $O(n)$ Space: $O(1)$

#32. Next Greater Element

#32 P1 Medium Stack & Queues

Problem: For every element of an array, find the first element to its right that is strictly greater than it. This element is called its next greater element. If no such element exists, the answer for that position is -1. Print the answers for all positions in the original order. A brute force search costs quadratic time. A monotonic stack makes it linear: scan from left to right, keep indices whose next greater element is still unknown in a stack with non-increasing values, and whenever the current element is larger than the top of the stack, pop the stack and record the current element as its answer.

Approach

- Use a monotonic decreasing stack of values, scanning from right to left.
- For each element, pop all stack values that are smaller or equal (they can never be a 'next greater' for anything to the left).
- Whatever remains on top of the stack is the next greater element; push the current element before moving on.

Solution (Java)

```

public int[] nextGreaterElement(int[] nums) {
    int n = nums.length;
    int[] res = new int[n];
    Deque<Integer> stack = new ArrayDeque<>(); // stores values
    for (int i = n - 1; i >= 0; i--) {
        while (!stack.isEmpty() && stack.peek() <= nums[i]) stack.pop();
        res[i] = stack.isEmpty() ? -1 : stack.peek();
        stack.push(nums[i]);
    }
    return res;
}

```

Time: $O(n)$ (each element pushed/popped at most once) Space: $O(n)$

#22. Implement Min Stack

#22

P1

Medium

Stack & Queues

Problem: Design a stack that supports the usual push, pop and top operations and can also report the minimum element currently stored in the stack, all in constant time. The command getMin returns the smallest value among the elements that are currently in the stack, and it must remain correct after any number of pops. A common trick is to store with every element the minimum of the stack up to that element, or to keep a second stack of minimums. Process the given commands in order and print the answer of each top and getMin command on its own line. Pop and top are never called on an empty stack.

Approach

- Use two stacks: the main stack for all pushed values, and a parallel 'min stack' that always keeps the current minimum on top.
- On push, also push the smaller of (new value, current min) onto the min stack.
- On pop, pop from both stacks together so they always stay in sync.

Solution (Java)

```

class MinStack {
    private Deque<Integer> stack = new ArrayDeque<>();
    private Deque<Integer> minStack = new ArrayDeque<>();

    public void push(int val) {
        stack.push(val);
        int currentMin = minStack.isEmpty() ? val : Math.min(val, minStack.peek());
        minStack.push(currentMin);
    }
    public void pop() { stack.pop(); minStack.pop(); }
    public int top() { return stack.peek(); }
    public int getMin() { return minStack.peek(); }
}

```

Time: $O(1)$ for every operation Space: $O(n)$

#20. Sliding Window Maximum

#20

P1

Hard

Stack & Queues

Problem: You are given an integer array and a window size k. The window starts at the leftmost k elements and slides one position to the right at each step until it reaches the end of the array. For every window position, determine the maximum element inside the window, and print all these maximums in order. A naive scan of each window costs $O(n \cdot k)$, which is too slow for large inputs. Use a monotonic deque that stores indices of candidates in decreasing order of their values, removing indices that fall out of the window from the front and smaller values from the back. The total time is linear.

Approach

- Use a deque (double-ended queue) that stores INDICES, kept in decreasing order of their values.

- For each new index: remove indices from the back whose value is smaller than the current value (they can never be the max again).
- Remove the front index if it has fallen outside the current window.
- The front of the deque is always the index of the window's maximum.

Solution (Java)

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> dq = new ArrayDeque<>(); // stores indices
    int n = nums.length;
    int[] res = new int[n - k + 1];
    for (int i = 0; i < n; i++) {
        while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst();
        while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) dq.pollLast();
        dq.offerLast(i);
        if (i >= k - 1) res[i - k + 1] = nums[dq.peekFirst()];
    }
    return res;
}
```

Time: $O(n)$ (each index pushed/popped once) Space: $O(k)$

BINARY SEARCH — 4 problems in reference bank

#24. Search in a 2D Matrix – II

#24

P1

Medium

Binary Search

Problem: You are given an m by n matrix in which every row is sorted in ascending order from left to right and every column is sorted in ascending order from top to bottom. Note that, unlike the simpler version, the first element of a row is not necessarily larger than the last element of the previous row. Decide whether a target value appears in the matrix and print true or false. A clever staircase search starts at the top right corner and moves left when the current value is larger than the target and down when it is smaller, achieving $O(m + n)$ time with constant extra space.

Approach

- Start at the TOP-RIGHT corner of the matrix (rows sorted left→right, columns sorted top→bottom).
- If the current value is greater than target, move left (eliminate a column). If smaller, move down (eliminate a row). If equal, found it.
- This eliminates one row or column at every step.

Solution (Java)

```
public boolean searchMatrix(int[][] matrix, int target) {
    int row = 0, col = matrix[0].length - 1;
    while (row < matrix.length && col >= 0) {
        if (matrix[row][col] == target) return true;
        else if (matrix[row][col] > target) col--;
        else row++;
    }
    return false;
}
```

Time: $O(m + n)$ **Space:** $O(1)$

#25. Find Peak Element

#25

P1

Medium

Binary Search

Problem: A peak element is an element that is strictly greater than its neighbours. Given an array in which no two adjacent elements are equal, find the index of any peak element. You may imagine that both ends of the array are surrounded by negative infinity, so the first or last element can be a peak if it beats its only neighbour. Your algorithm should run in $O(\log n)$ time: compare the middle element with its right neighbour and move toward the side that is rising, because a peak is guaranteed there. Any valid peak index is accepted by the checker; the expected outputs below show the smallest such index.

Approach

- Binary search works here because 'always going toward the larger neighbour' is guaranteed to reach a peak (array edges count as -infinity).
- If mid's right neighbour is greater, a peak must exist to the right — search right half (including mid+1). Otherwise search left half (including mid).

Solution (Java)

```
public int findPeakElement(int[] nums) {
    int lo = 0, hi = nums.length - 1;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (nums[mid] < nums[mid + 1]) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}
```

Time: $O(\log n)$ **Space:** $O(1)$

#26. Median of Two Sorted Arrays

#26

P1

Hard

Binary Search

Problem: Given two sorted arrays a and b of sizes n and m, find the median of the combined sorted data. If the total count is odd, the median is the middle element; if it is even, the median is the average of the two middle elements. Merging the arrays takes $O(n + m)$ time, but the intended solution is $O(\log(\min(n, m)))$ and applies binary search on the shorter array to find a partition where every element on the left side is not larger than every element on the right side. Print the median as a decimal number with exactly one digit after the decimal point.

Approach

- Binary search on the SMALLER array to find a partition point such that: elements on the left of both partitions combined equal elements on the right combined (or off by one for odd total length).
- A valid partition satisfies: $\text{maxLeftA} \leq \text{minRightB}$ and $\text{maxLeftB} \leq \text{minRightA}$.
- Once found, the median is derived from these four boundary values depending on whether the total length is even or odd.

Solution (Java)

```
public double findMedianSortedArrays(int[] a, int[] b) {
    if (a.length > b.length) return findMedianSortedArrays(b, a); // ensure a is smaller
    int n1 = a.length, n2 = b.length;
    int lo = 0, hi = n1;
    while (lo <= hi) {
        int cutA = (lo + hi) / 2;
        int cutB = (n1 + n2 + 1) / 2 - cutA;
        int maxLeftA = (cutA == 0) ? Integer.MIN_VALUE : a[cutA - 1];
        int minRightA = (cutA == n1) ? Integer.MAX_VALUE : a[cutA];
        int maxLeftB = (cutB == 0) ? Integer.MIN_VALUE : b[cutB - 1];
        int minRightB = (cutB == n2) ? Integer.MAX_VALUE : b[cutB];
        if (maxLeftA <= minRightB && maxLeftB <= minRightA) {
            if ((n1 + n2) % 2 == 0)
                return (Math.max(maxLeftA, maxLeftB) + Math.min(minRightA, minRightB)) / 2.0;
            else
                return Math.max(maxLeftA, maxLeftB);
        } else if (maxLeftA > minRightB) hi = cutA - 1;
        else lo = cutA + 1;
    }
    throw new IllegalArgumentException("Input arrays are not sorted");
}
```

Time: $O(\log(\min(n, m)))$ Space: $O(1)$

#88. Search X in a Sorted Array

#88

P2

Medium

Binary Search

Problem: You are given an array of distinct integers sorted in ascending order and a target value x. Print the zero-based index of x in the array, or -1 if x does not occur. A linear scan works but takes $O(n)$ time, so use binary search instead: repeatedly compare x with the middle element of the current range, return its index when they are equal, and discard the left or the right half depending on the comparison. Take care to compute the middle index as $\text{low} + (\text{high} - \text{low}) / 2$ to avoid integer overflow in languages with fixed size integers. Report -1 when the range becomes empty without finding the value.

Approach

- Standard binary search: repeatedly halve the search range based on comparing the middle element to the target.

Solution (Java)

```
public int search(int[] nums, int target) {
    int lo = 0, hi = nums.length - 1;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (nums[mid] == target) return mid;
        else if (nums[mid] < target) lo = mid + 1;
        else hi = mid - 1;
    }
    return -1;
}
```

Time: $O(\log n)$ Space: $O(1)$

SORTING — 2 problems in reference bank

#14. Sort an Array of 0's, 1's and 2's

#14 P1 Medium Sorting

Problem: You are given an array that contains only the values 0, 1 and 2. Sort the array in ascending order without using a general sorting function. This is the well known Dutch National Flag problem. It can be solved in a single pass with three pointers: low marks the boundary of zeros, high marks the boundary of twos, and mid scans the unknown region, swapping elements to the correct side. The algorithm runs in linear time and uses constant extra space. Print the sorted array as space separated integers on one line. The array may already be sorted or may contain only one distinct value.

Approach

- Dutch National Flag algorithm — three pointers: low (boundary of 0s), mid (current/unknown), high (boundary of 2s).
- If `nums[mid]` is 0, swap with low, advance both low and mid.
- If it's 1, just advance mid.
- If it's 2, swap with high and decrement high (don't advance mid, since the swapped-in value is still unchecked).

Solution (Java)

```
public void sortColors(int[] nums) {
    int low = 0, mid = 0, high = nums.length - 1;
    while (mid <= high) {
        if (nums[mid] == 0) {
            int t = nums[low]; nums[low] = nums[mid]; nums[mid] = t;
            low++; mid++;
        } else if (nums[mid] == 1) {
            mid++;
        } else {
            int t = nums[mid]; nums[mid] = nums[high]; nums[high] = t;
            high--;
        }
    }
}
```

Time: $O(n)$ (single pass) **Space:** $O(1)$

#48. Find the Repeating and Missing Number

#48 P1 Medium Sorting

Problem: You are given an array of size n that should contain every integer from 1 to n exactly once, but one number appears twice and one number is missing. Find both the repeating number and the missing number. Print them on one line, first the repeating value and then the missing value. You can sort the array and compare each element with its expected value, use a frequency array, or apply mathematics: the difference between the actual sum and the expected sum of $1..n$, and the difference between the actual and expected sums of squares, form two equations that reveal both numbers directly without extra space.

Approach

- Let 'missing' = m and 'repeating' = r . Compute `expectedSum` ($1..n$) and `expectedSumSq` ($1^2..n^2$), and the array's `actualSum` and `actualSumSq`.
- $\text{diff1} = \text{expectedSum} - \text{actualSum} = m - r$.
- $\text{diff2} = \text{expectedSumSq} - \text{actualSumSq} = m^2 - r^2 = (m - r)(m + r)$, so $(m + r) = \text{diff2} / \text{diff1}$.
- Solve the two linear equations for m and r .

Solution (Java)

```
public int[] findMissingAndRepeating(int[] nums) {
    int n = nums.length;
    long expSum = (long) n * (n + 1) / 2;
    long expSumSq = (long) n * (n + 1) * (2L * n + 1) / 6;
    long actSum = 0, actSumSq = 0;
    for (int x : nums) { actSum += x; actSumSq += (long) x * x; }
    long diff1 = expSum - actSum;           // missing - repeating
    long diff2 = expSumSq - actSumSq;       // missing^2 - repeating^2
    long sum = diff2 / diff1;               // missing + repeating
    long missing = (diff1 + sum) / 2;
    long repeating = sum - missing;
    return new int[]{(int) repeating, (int) missing};
}
```

Time: O(n) Space: O(1)

BINARY SEARCH TREES — 3 problems in reference bank

#15. Check if a Binary Tree is a BST

#15 P1 Medium Binary Search Trees

Problem: Given the root of a binary tree, determine whether it is a valid binary search tree. In a valid BST, every node's left subtree contains only values strictly smaller than the node, every node's right subtree contains only values strictly greater than the node, and both subtrees are themselves valid BSTs. Comparing a node only with its direct children is not enough, because a deeper descendant can still violate an ancestor's bound. Pass a permitted range down the recursion, or verify that an inorder traversal is strictly increasing. Print true if the tree is a valid BST and false otherwise.

Approach

- A tree is a valid BST only if EVERY node lies within a valid (min, max) range dictated by its ancestors — not just compared to its immediate parent.
- Recursively pass down a shrinking (min, max) bound: going left tightens the max to the parent's value; going right tightens the min.

Solution (Java)

```
public boolean isValidBST(TreeNode root) {
    return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
}
private boolean validate(TreeNode node, long min, long max) {
    if (node == null) return true;
    if (node.val <= min || node.val >= max) return false;
    return validate(node.left, min, node.val) && validate(node.right, node.val, max);
}
```

Time: $O(n)$ Space: $O(h)$ (h = tree height, recursion stack)

#37. Lowest Common Ancestor in a BST

#37 P1 Medium Binary Search Trees

Problem: Given a binary search tree and two nodes p and q that are present in it, find their lowest common ancestor, meaning the deepest node that has both p and q among its descendants. A node is allowed to be a descendant of itself, so if p is an ancestor of q, then p is the answer. Use the ordering property of the BST instead of exploring the whole tree: starting from the root, if both values are smaller than the current node move to the left child, if both are larger move to the right child, and otherwise the current node is the split point and hence the answer. Print the value of that node.

Approach

- In a BST, use the value-ordering property directly instead of general tree LCA logic.
- Starting at the root: if both p and q are smaller than the current node, go left; if both are larger, go right; otherwise, the current node is the LCA (it's the split point).

Solution (Java)

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    TreeNode cur = root;
    while (cur != null) {
        if (p.val < cur.val && q.val < cur.val) cur = cur.left;
        else if (p.val > cur.val && q.val > cur.val) cur = cur.right;
        else return cur;
    }
    return null;
}
```

Time: $O(h)$ Space: $O(1)$ (iterative)

#27. Kth Smallest and Kth Largest Element in a BST

#27

P1

Medium

Binary Search Trees

Problem: Given the root of a binary search tree and an integer k , find both the k -th smallest and the k -th largest values stored in the tree. Because an inorder traversal of a BST visits values in ascending order, the k -th smallest value is the k -th node visited in that traversal, while the k -th largest is found by traversing in reverse inorder (right, node, left) or as the $(n - k + 1)$ -th smallest. Try to stop the traversal as soon as the answer is found. Print the two numbers on one line, first the k -th smallest and then the k -th largest, separated by a single space.

Approach

- In-order traversal of a BST visits nodes in ascending sorted order.
- Do an iterative in-order traversal using an explicit stack; the k -th node visited is the k -th smallest.
- For k -th largest, do the same traversal in reverse order (right, node, left).

Solution (Java)

```
public int kthSmallest(TreeNode root, int k) {
    Deque<TreeNode> stack = new ArrayDeque<>();
    TreeNode cur = root;
    while (true) {
        while (cur != null) { stack.push(cur); cur = cur.left; }
        cur = stack.pop();
        if (--k == 0) return cur.val;
        cur = cur.right;
    }
}
```

Time: $O(h + k)$ **Space:** $O(h)$

GREEDY ALGORITHMS — 3 problems in reference bank

#16. Jump Game – I

#16

P1

Medium

Greedy Algorithms

Problem: You are standing at the first index of an array of non-negative integers. Each element tells you the maximum number of positions you can jump forward from that index; you may jump any smaller distance too. Decide whether you can reach the last index. A greedy strategy works in linear time: keep track of the farthest index reachable so far while scanning from left to right, and if the current index is ever beyond that farthest reach, you are stuck. Print true if the last index can be reached and false otherwise. A single element array is trivially reachable.

Approach

- Greedy: track the furthest index reachable so far ('maxReach') as you scan left to right.
- If the current index ever exceeds maxReach, you can never get there — return false.
- Otherwise update $\text{maxReach} = \max(\text{maxReach}, i + \text{nums}[i])$ and continue; if you finish the scan, the last index is reachable.

Solution (Java)

```
public boolean canJump(int[] nums) {
    int maxReach = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > maxReach) return false;
        maxReach = Math.max(maxReach, i + nums[i]);
    }
    return true;
}
```

Time: $O(n)$ Space: $O(1)$

#33. Meeting Rooms

#33

P1

Medium

Greedy Algorithms

Problem: You are given the start and end times of several meetings, each represented as an interval [start, end). Determine whether one person can attend all of the meetings. A person can move directly from one meeting to another if the earlier meeting ends at exactly the same time the next one starts, so touching intervals do not conflict. Sort the intervals by start time and check that every meeting starts no earlier than the previous meeting ends. Any overlap means the person cannot attend all meetings. Print true if all meetings can be attended without overlaps, and false otherwise.

Approach

- This asks for the minimum number of rooms needed so no two overlapping meetings share a room.
- Split into two sorted arrays: all start times and all end times.
- Walk through with two pointers: whenever the next start time is before the next end time, one more room is needed (increment a counter); otherwise a room frees up (decrement). Track the maximum concurrent rooms.

Solution (Java)

```

public int minMeetingRooms(int[][] intervals) {
    int n = intervals.length;
    int[] starts = new int[n], ends = new int[n];
    for (int i = 0; i < n; i++) { starts[i] = intervals[i][0]; ends[i] = intervals[i][1]; }
    Arrays.sort(starts);
    Arrays.sort(ends);
    int rooms = 0, maxRooms = 0, s = 0, e = 0;
    while (s < n) {
        if (starts[s] < ends[e]) { rooms++; s++; }
        else { rooms--; e++; }
        maxRooms = Math.max(maxRooms, rooms);
    }
    return maxRooms;
}

```

Time: $O(n \log n)$ **Space:** $O(n)$

#29. Valid Parenthesis Checker (with Wildcard)

#29

P1

Medium

Greedy Algorithms

Problem: Given a string containing only the characters '(', ')' and '*', check whether it can be considered valid. Every '(' must have a matching ')' that comes after it, every ')' must have a matching '(' before it, and the character '*' can be treated as '(' , as ')' or as an empty string. The greedy technique tracks the smallest and largest possible number of unmatched opening brackets while scanning: a star widens the range, an opening bracket increases both bounds, and a closing bracket decreases both. If the maximum ever becomes negative the string is invalid. Print true or false.

Approach

- Track a RANGE [low, high] of how many unmatched open brackets are possible, instead of one fixed count.
- For '(' both low and high increase; for ')' both decrease; for '*' low decreases (treat as ')') and high increases (treat as '(').
- If high ever goes negative, too many ')' were forced — invalid. If low goes negative, clamp it to 0 (that '*' can act as empty). At the end, the string is valid only if low can be 0.

Solution (Java)

```

public boolean checkValidString(String s) {
    int low = 0, high = 0;
    for (char c : s.toCharArray()) {
        if (c == '(') { low++; high++; }
        else if (c == ')') { low--; high--; }
        else { low--; high++; } // '*'
        if (high < 0) return false;
        if (low < 0) low = 0;
    }
    return low == 0;
}

```

Time: $O(n)$ **Space:** $O(1)$

DYNAMIC PROGRAMMING — 2 problems in reference bank

#167. Best Time to Buy and Sell Stock

#167	P1	Medium	Dynamic Programming
------	----	--------	---------------------

Problem: You are given an array prices where prices[i] is the price of a given stock on day i. You want to maximise your profit by choosing a single day to buy one share and a different, later day to sell that share. Return the maximum profit you can achieve from this single transaction. If no profit is possible because the price never rises after a lower price, return 0. You cannot sell before you buy. Traverse the prices once while keeping the minimum price seen so far; for each day, the best profit if you sell today is the current price minus that minimum. Keep the maximum of these values. The solution runs in linear time using constant extra space.

Approach

- Track the minimum price seen so far while scanning left to right.
- At each day, the best possible profit if selling today is (today's price – minimum price so far). Keep the running maximum of that value.

Solution (Java)

```
public int maxProfit(int[] prices) {
    int minPrice = Integer.MAX_VALUE, maxProfit = 0;
    for (int p : prices) {
        minPrice = Math.min(minPrice, p);
        maxProfit = Math.max(maxProfit, p - minPrice);
    }
    return maxProfit;
}
```

Time: O(n) Space: O(1)

#168. Best Time to Buy and Sell Stock II

#168	P1	Medium	Dynamic Programming
------	----	--------	---------------------

Problem: You are given an array prices where prices[i] is the price of a stock on day i. On each day you may decide to buy and or sell the stock, but you can hold at most one share at any time. You are allowed to buy and sell on the same day. Find and return the maximum profit you can achieve when the number of transactions is unlimited. The greedy insight is that every rise between two consecutive days can be captured separately: if prices[i] is greater than prices[i - 1], add the difference to the profit. The sum of all positive daily differences equals the best possible total. The algorithm makes one pass over the array and needs constant extra space.

Approach

- Since unlimited transactions are allowed, you can capture every upward price movement.
- Simply sum up every positive difference between consecutive days — this equals the maximum total profit achievable.

Solution (Java)

```
public int maxProfit(int[] prices) {
    int profit = 0;
    for (int i = 1; i < prices.length; i++) {
        if (prices[i] > prices[i - 1]) profit += prices[i] - prices[i - 1];
    }
    return profit;
}
```

Time: O(n) Space: O(1)

TRIES — 2 problems in reference bank

#171. Longest Common Prefix

#171 P1 Medium Tries

Problem: Given a list of n strings, find the longest string that is a prefix of every string in the list. A prefix of a string is a leading part of it, such as `fl` for `flower`. If the strings have no common prefix, print `-1` instead of an empty string. For the strings `flower`, `flow` and `flight`, the longest common prefix is `fl`. One approach compares the first string with each other string and shortens the candidate prefix until it matches. A Trie gives another method: insert all the strings and walk down from the root while the current node has exactly one child and no string ends there; the characters on that walk form the prefix.

Approach

- Vertical scanning: compare characters column by column across all strings simultaneously.
- For each character position, check if all strings agree at that position; stop at the first mismatch or the first string that ends.

Solution (Java)

```
public String longestCommonPrefix(String[] strs) {
    if (strs.length == 0) return "";
    for (int i = 0; i < strs[0].length(); i++) {
        char c = strs[0].charAt(i);
        for (int j = 1; j < strs.length; j++) {
            if (i == strs[j].length() || strs[j].charAt(i) != c) {
                return strs[0].substring(0, i);
            }
        }
    }
    return strs[0];
}
```

Time: $O(n \cdot m)$ (n strings, m = shortest length) **Space:** $O(1)$

#52. Trie Implementation and Advanced Operations

#52 P1 Hard Tries

Problem: Implement a trie, also called a prefix tree, that stores lowercase words and supports the following commands: `insert(word)`, `erase(word)`, `search(word)`, `startsWith(prefix)`, `countWords(word)` and `countPrefix(prefix)`. Words may be inserted several times, and each insertion counts separately. Erasing removes one occurrence and does nothing if the word is absent. `search` and `startsWith` answer true or false, while `countWords` returns how many copies of the exact word are stored and `countPrefix` returns how many stored words start with the given prefix, counting duplicates. Keep two counters in every trie node to support this efficiently. Print the result of each query command.

Approach

- Each `TrieNode` holds an array of 26 children (one per lowercase letter) and a boolean flag marking the end of a word.
- `insert()`: walk/create nodes character by character, mark the final node as a word-end.
- `search()`: walk the exact path; return true only if it fully matches AND the final node is marked as a word-end.
- `startsWith()`: same walk, but return true as soon as the full prefix path exists (no need to check the word-end flag).

Solution (Java)

```

class Trie {
    class Node { Node[] children = new Node[26]; boolean isEnd = false; }
    private Node root = new Node();

    public void insert(String word) {
        Node cur = root;
        for (char c : word.toCharArray()) {
            int i = c - 'a';
            if (cur.children[i] == null) cur.children[i] = new Node();
            cur = cur.children[i];
        }
        cur.isEnd = true;
    }
    public boolean search(String word) {
        Node node = find(word);
        return node != null && node.isEnd;
    }
    public boolean startsWith(String prefix) {
        return find(prefix) != null;
    }
    private Node find(String s) {
        Node cur = root;
        for (char c : s.toCharArray()) {
            int i = c - 'a';
            if (cur.children[i] == null) return null;
            cur = cur.children[i];
        }
        return cur;
    }
}

```

Time: $O(\text{word length})$ per operation Space: $O(\text{total characters inserted})$

MATHEMATICS — 5 problems in reference bank

#75. Armstrong Number

#75 P2 Easy Mathematics

Problem: A positive integer is an Armstrong number, also known as a narcissistic number, if it equals the sum of its own digits each raised to the power of the number of digits. For example, 153 has three digits and $1^3 + 5^3 + 3^3 = 153$, so it is an Armstrong number, while 154 is not. Given an integer n , determine whether it is an Armstrong number. Extract the digits by repeatedly taking the remainder and integer division by ten, count how many digits there are, and then accumulate the powered digits. Print true if n is an Armstrong number and false otherwise. Every single digit number is an Armstrong number.

Approach

- An Armstrong number equals the sum of its own digits, each raised to the power of the total digit count.
- Count the digits, then sum $\text{digit}^{\text{numDigits}}$ for every digit; compare to the original number.

Solution (Java)

```
public boolean isArmstrong(int num) {
    int digits = String.valueOf(num).length();
    int sum = 0, temp = num;
    while (temp > 0) {
        int d = temp % 10;
        sum += Math.pow(d, digits);
        temp /= 10;
    }
    return sum == num;
}
```

Time: $O(\log_{10}(\text{num}))$ Space: $O(1)$

#184. Fibonacci Series

#184 P2 Easy Mathematics

Problem: Given an integer n , print the first n terms of the Fibonacci series. The series starts with 0 and 1, and every next term is the sum of the previous two terms, so the series is 0, 1, 1, 2, 3, 5, 8 and so on. If n is 1, print only 0. The values are separated by single spaces in the output. An iterative solution keeps two variables for the last two terms, prints the current term, and then updates the variables in a loop that runs n times, which takes linear time and constant extra space. A recursive solution is shorter but recomputes the same terms many times and takes exponential time unless it is memoised.

Approach

- Iterative approach using two variables to hold the last two Fibonacci numbers — avoids the exponential blow-up of naive recursion.

Solution (Java)

```
public int fib(int n) {
    if (n <= 1) return n;
    int prev = 0, curr = 1;
    for (int i = 2; i <= n; i++) {
        int next = prev + curr;
        prev = curr;
        curr = next;
    }
    return curr;
}
```

Time: $O(n)$ Space: $O(1)$

#185. Swap Two Numbers Without a Temporary Variable

#185

P2

Easy

Mathematics

Problem: Given two integers a and b, swap their values without using a third temporary variable, and print the values after the swap in the order a followed by b. For example, if a is 5 and b is 9, the output is 9 5. There are two classic methods. The arithmetic method sets $a = a + b$, then $b = a - b$, then $a = a - b$. The bitwise method uses exclusive OR: $a = a \oplus b$, $b = a \oplus b$, $a = a \oplus b$. Both methods work with negative numbers and with equal values, but the arithmetic method may overflow a fixed-size integer type when the sum is very large, so the XOR method is safer in languages such as C and Java.

Approach

- Arithmetic trick: add the two numbers into 'a', then derive 'b' and 'a' from that combined sum — no temp variable needed.
- (A XOR-based swap is an alternative that also avoids overflow concerns with very large numbers.)

Solution (Java)

```
int a = 5, b = 10;
a = a + b; // a now holds the sum
b = a - b; // b becomes original a
a = a - b; // a becomes original b
// Now a and b are swapped
```

Time: O(1) Space: O(1)

#97. Check Whether a Given Number Is Even or Odd

#97

P2

Easy

Mathematics

Problem: You are given an integer n, which may be negative or zero. Print Even if n is divisible by 2 and Odd otherwise. Zero is considered an even number. This is one of the simplest programming exercises, but it still contains a few traps. The remainder operator can return a negative value for negative inputs in languages such as C, C++ and Java, so testing whether $n \% 2$ equals 1 would wrongly treat -3 as even; test whether $n \% 2$ is zero, or use the bitwise operation $n \& 1$ which works for negative numbers in two's complement form. Print exactly Even or Odd with the first letter capitalised.

Approach

- A number is even if it's exactly divisible by 2 — check using the modulus operator, or the faster bitwise AND with 1 (0 means even).

Solution (Java)

```
public boolean isEven(int n) {
    return (n & 1) == 0; // equivalent to n % 2 == 0
}
```

Time: O(1) Space: O(1)

#181. Dealership Tyres

#181

P2

Easy

Mathematics

Problem: A vehicle dealership has sold some cars and some bikes. Every car has four tyres and every bike has two tyres. Given the number of cars and the number of bikes, calculate and print the total number of tyres of all the vehicles. Spare tyres are not counted. For example, a dealership with 3 cars and 2 bikes has $3 \times 4 + 2 \times 2 = 16$ tyres. The problem only tests basic arithmetic and input handling, so the solution is a single expression: multiply the number of cars by four, multiply the number of bikes by two, and add the two products. Use a 64-bit integer type when the counts can be large, because four times a big count may exceed the range of a 32-bit integer.

Approach

- For each dealership, total tyres = (number of cars $\times$ 4) + (number of bikes $\times$ 2).
- Read each dealership's (cars, bikes) pair and print the computed total.

Solution (Java)

```
Scanner sc = new Scanner(System.in);
int dealerships = sc.nextInt();
for (int i = 0; i < dealerships; i++) {
    int cars = sc.nextInt();
    int bikes = sc.nextInt();
    int totalTyres = cars * 4 + bikes * 2;
    System.out.println(totalTyres);
}
```

Time: $O(1)$ per dealership Space: $O(1)$

Quick Pattern Cheat-Sheet

When you hit an unsolved problem from the index and aren't sure where to start, match it to one of these patterns first:

If the problem involves...	Try this pattern
Finding pairs/subarrays with a target sum	HashMap of value→index or prefix sums
Longest/shortest substring or subarray with a condition	Sliding window (two pointers, expand/shrink)
Sorted array, searching for a value/boundary/peak	Binary search
Next greater/smaller element, span problems	Monotonic stack
Window maximum/minimum as it slides	Monotonic deque
Overlapping intervals, scheduling, 'minimum resources'	Sort by start/end + greedy sweep, or min-heap
BST-specific (search/insert/LCA/kth element)	Use BST ordering property — no need for general tree algorithms
Prefix matching, word dictionary problems	Trie (prefix tree)
Buy/sell stock, cooldowns, transaction fees	Dynamic programming with 'state' = holding or not holding stock
Counting frequencies, anagrams, grouping	HashMap keyed by sorted string or frequency signature
0s/1s/2s sorting, partitioning by category	Dutch National Flag (three pointers)
Missing/repeating number in 1..n	Sum & sum-of-squares equations, or XOR trick

Remember: in the real test, a correct brute-force solution that passes most test cases beats an incomplete optimized one. If you can't recall the optimal pattern under time pressure, write the straightforward $O(n^2)$ version first, get it working, then optimize if time remains.